HTML & CSS Self-Study Guide

ITWS1100 - Intro to Information Technology & Web Science

Class cancelled tomorrow (snow day). Use this guide to prepare for Thursday's combined lecture/lab.

Topic Overview

This unit covers **HTML (HyperText Markup Language)** and **CSS (Cascading Style Sheets)** — the foundational technologies for creating web pages.

The Big Picture

HTML provides the *structure* and *content* of a web page using tags.
CSS provides the *presentation* and *styling* of that content.

Why Learn This?

- HTML is a **semantic markup** — understanding it helps you make good decisions about how machines (like search engines) interpret your documents
- If you do any web coding or design real-world web applications, you need to be fluent in HTML & CSS
- These are fundamental skills for *any* career in IT, web development, or digital media

Key Concepts to Understand

1. HTML Document Structure

Every HTML5 document follows this pattern:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    Content goes here
  </body>
</html>
```

- `<!DOCTYPE html>` — Tells the browser this is HTML5
- `<head>` — Contains metadata, title, CSS links, scripts (not visible on page)
- `<body>` — Contains all visible content

2. Common HTML Elements

Element	Purpose	Display Type
`<h1>` to `<h6>`	Headings (h1 = most important)	Block
`<p>`	Paragraphs	Block
`<div>`	Generic container for grouping	Block
`<ul>`, `<ol>`, `<li>`	Lists (unordered, ordered, list item)	Block
`<em>`, `<strong>`	Emphasis (italic, bold)	Inline

Element	Purpose	Display Type
`<a href="...">`	Links	Inline
`<img src="..." alt="...">`	Images	Inline

3. Block vs. Inline Elements

- **Block elements**: Take full width available, create line breaks before and after (paragraphs, headings, divs)
- **Inline elements**: Flow with surrounding content, no line breaks (emphasized text, links, images)

4. CSS Basics

CSS rules follow this structure:

```
selector {
  property: value;
  property: value;
}
```

THREE WAYS TO ADD CSS

Method	Example	When to Use
External (best)	`<link href="style.css" rel="stylesheet">`	Multi-page sites, production
Embedded	`<style>...</style>` in head	Single-page, portability
Inline (avoid)	`<p style="color: red;">`	Quick testing only

5. CSS Selectors

Selector Type	Example	Selects
Element	`p { }`	All `<p>` elements
Class	`.highlight { }`	Elements with `class="highlight"`
ID	`#footer { }`	Element with `id="footer"`
Combined	`p.intro { }`	`<p>` elements with `class="intro"`
Descendant	`div p { }`	`<p>` elements inside a `<div>`

Important: IDs vs. Classes

IDs must be unique — only one element per page can have a specific ID.
Classes can be reused — multiple elements can share the same class.

6. CSS Specificity (The Cascade)

When rules conflict, more specific wins:

1. **ID selectors** (#footer) beat class selectors
2. **Class selectors** (.highlight) beat element selectors
3. **Element selectors** (p) are least specific
4. Later rules override earlier rules (when specificity is equal)
5. Inline styles override embedded/external

7. The CSS Box Model

Every element is a box with four parts (from inside to outside):

1. **Content** — The actual text/images
2. **Padding** — Space inside the border (around content)
3. **Border** — The edge around the padding
4. **Margin** — Space outside the border (between elements)

```css
.box {
  padding: 20px;          /* all sides */
  margin: 10px 20px;       /* top/bottom, left/right */
  border: 1px solid black;
}
```

8. CSS Sizes & Colors

COMMON SIZE UNITS

- `em` — Relative to font size (scalable, great for responsive design)
- `px` — Pixels (precise, good for fixed layouts)
- `%` — Percentage of parent element

COLORS (HEXADECIMAL)

Colors use hex values: `#RRGGBB` where each pair is 00-FF

- `#FF0000` = Red (shorthand: `#F00`)
- `#00FF00` = Green (shorthand: `#0F0`)
- `#0000FF` = Blue (shorthand: `#00F`)
- `#000000` = Black, `#FFFFFF` = White

9. Flexbox (Modern Layout)

Flexbox makes layout much easier than floats:

```css
.container {
  display: flex;
  justify-content: space-between;  /* horizontal alignment */
  align-items: center;           /* vertical alignment */
}

.item {
  flex: 1;  /* items grow to fill space equally */
}
```

Pro Tip

For the resume lab, Flexbox is excellent for laying out sections with dates aligned to the right. See the sample templates provided!

Required Self-Study Resources

Complete these tutorials before Thursday's class:

W3Schools HTML Tutorial

Basic Tags, Attributes, Headings, Paragraphs, Links, Lists

30-45 minutes

W3Schools CSS Tutorial

Syntax, Selectors, Box Model, basic Flexbox

30-45 minutes

web.dev: CSS Selectors

Excellent deep-dive into CSS selectors with interactive examples

20-30 minutes

W3Schools Flexbox

Modern layout technique you'll use in the resume lab

15-20 minutes

Reference Materials (Skim/Bookmark)

HTML Tag Reference

Complete list of HTML tags organized by function. Ignore deprecated tags.

Reference

CSS Selectors Reference

Quick reference for all CSS selector types

Reference

Recommended Additional Resources

web.dev Learn CSS

Comprehensive, modern CSS course from Google. Excellent for deeper understanding.

Full Course

MDN Web Docs

The definitive reference for web technologies. More detailed than W3Schools.

Reference

CSS-Tricks Flexbox Guide

Visual guide to every Flexbox property. Great for reference.

Reference

CSS-Tricks Selectors

Another excellent explanation of CSS selectors with examples

15-20 minutes

Chrome DevTools

Learn to use your browser's Developer Tools (F12) to inspect HTML & CSS. Search YouTube for "Chrome DevTools HTML CSS tutorial" — The Net Ninja has good introductory videos.

Lab Preview: HTML Resume

On Thursday, you will build an HTML resume. Here's what to expect:

What You'll Create

- A single HTML file named `yourRCSID-FullName-resume.html`
- Sections: Contact Info, Objective/Profile, Education, Skills, Experience, Activities
- Styled with CSS (embedded or external)

Technical Requirements

- Valid HTML5 — test at validator.w3.org
- Valid CSS — test at jigsaw.w3.org/css-validator
- **NO tables for layout** — use semantic HTML + CSS (Flexbox or Grid encouraged)
- Proper indentation (2-3 spaces per level)

Grading Rubric (100 points)

Criteria	Points	Description
Validity & Code Style	**30**	Does the page validate? Is indentation consistent and readable?
Completeness	**50**	Is the resume complete? Are all major sections represented with real content?
Quality & Creativity	**20**	How well does it match professional standards? Is the design thoughtful and creative?

Career Center Review

After submitting your lab, you'll take your resume to the Career Center for review — so make it good! This is a real document you can use for internships and jobs.

Practice Exercise: Warm-Up Before Thursday

This exercise reinforces your DevOps workflow from Lab 1 (branch, code, commit, push, deploy) while giving you hands-on practice with HTML & CSS before the resume lab.

Goal

Create a simple "About Me" page to practice HTML structure, CSS styling, and your Git workflow. This is **not graded** — it's practice to get comfortable before Thursday.

1 **Create a New Branch**

Open **GitHub Desktop** and make sure you're in your `iit` repository.
Click **Current Branch** → **New Branch** → name it `practice-aboutme`
This keeps your practice separate from your main work (just like we'll do with `lab02` on Thursday).

2 **Create Your Practice File**

In VS Code, create a new folder: `iit/practice/`
Inside that folder, create a file: `aboutme.html`

3 **Write Basic HTML**

Type out this structure (don't copy-paste — typing helps you learn!):

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>About Me - Your Name</title>
   <style>
      /* Your CSS will go here */
   </style>
</head>
<body>
   <h1>Your Name</h1>
   <p>A short introduction about yourself.</p>

   <h2>My Interests</h2>
   <ul>
      <li>Interest 1</li>
      <li>Interest 2</li>
      <li>Interest 3</li>
```

```
    </ul>

    <h2>Contact</h2>
    <p>Email: <a href="mailto:yourRCS@rpi.edu">yourRCS@rpi.edu</a></p>
</body>
</html>
```

4 Add Some CSS

Inside the `<style>` tag, add these rules:

```
body {
    font-family: Arial, sans-serif;
    max-width: 600px;
    margin: 40px auto;
    padding: 20px;
    background-color: #f5f5f5;
}

h1 {
    color: #c41230;  /* RPI Red! */
}

h2 {
    border-bottom: 2px solid #333;
    padding-bottom: 5px;
}

a {
    color: #c41230;
}
```

5 Test Locally

In VS Code, right-click your `aboutme.html` file and select **Open with Live Server** (or use the Live Preview extension).

Your browser should open and show your styled page. Make changes and watch them update live!

6 **Experiment!**

Try these modifications to build your CSS intuition:

- Change the background color to something else
- Add `padding` and `margin` to different elements
- Try `text-align: center;` on the h1
- Add a class to style one list item differently

7 **Commit & Push**

In **GitHub Desktop**:

1. You should see your new files in the "Changes" panel
2. Write a commit message: `Add practice aboutme page`
3. Click **Commit to practice-aboutme**
4. Click **Publish branch** (or Push origin) to send it to GitHub

8 **Deploy to Your Server (Optional)**

If you want full practice:

1. Start your Azure instance
2. SSH into your web server
3. Navigate to your `iit` folder
4. `git fetch origin`
5. `git checkout practice-aboutme`
6. `git pull origin practice-aboutme`
7. Test at `FQDN/iit/practice/aboutme.html`

You Did It!

You just practiced the entire DevOps workflow: **branch** → **code** → **test** → **commit** → **push** → **deploy**. This is exactly what you'll do for Lab 2 on Thursday!

Tools You Need Ready

- ☐ **VS Code** with Live Preview extension installed (`ms-vscode.live-server`)
- ☐ **Chrome or Edge** browser with Developer Tools (press F12)
- ☐ **GitHub Desktop** connected to your `iit` repository
- ☐ Your repository **synced** between GitHub.com and your local machine

Come to Thursday's Lab Ready With:

1. **Questions** about HTML/CSS concepts from your self-study
2. A rough idea of your **resume content** (education, skills, experience, activities)
3. Your repo ready to go: **Create a new branch called** `lab02` and make sure it's synced between GitHub and your local machine. (Your Azure instances can wait until deployment time.)

Thursday's Plan

We'll start with a brief Q&A to address any confusion from self-study, then dive straight into building your resumes. Come prepared so we can maximize hands-on coding time!